

Sistema de Consulta Eficiente com Tabela Hash e Filtro de Bloom

Samara da Silva Gonçalves | Maria Raquel Nunes Silva

Pau dos Ferros-RN | 2026

Contexto e Objetivos

O Desafio do Big Data

Em sistemas modernos, a busca em grandes volumes de dados exige estruturas que equilibrem **velocidade** e **uso de recursos**.

Consultas a elementos inexistentes em uma base de dados podem gerar latência desnecessária ao acessar a memória principal ou disco.

A Proposta do Projeto

- **Otimização:** Reduzir o tempo de busca interceptando consultas inúteis.
- **Hibridismo:** Unir a precisão da Tabela Hash com a velocidade probabilística do Filtro de Bloom.
- **Escalabilidade:** Validar o comportamento do sistema em cargas de 1.000 a 100.000 registros.

Arquitetura da Tabela Hash

A Tabela Hash é a nossa estrutura **determinística**. Ela garante que, se um elemento existe, ele será encontrado em tempo constante.

- **Algoritmo djb2:** Escolhido pela sua alta eficiência em espalhar strings uniformemente, minimizando a formação de "clusters" de dados.
- **Tratamento de Colisões:** Implementado via **Encadeamento Externo**. Cada índice da tabela aponta para o início de uma lista encadeada.

Dimensionamento e Carga

O tamanho da tabela foi definido como **$2N$** (dobro da carga esperada).

Fator de Carga $\alpha = 0,5$

Resultado: Graças a este dimensionamento, obtivemos **0 colisões** em todas as baterias de testes, mantendo a complexidade em $O(1)$

Conceitos do Filtro de Bloom

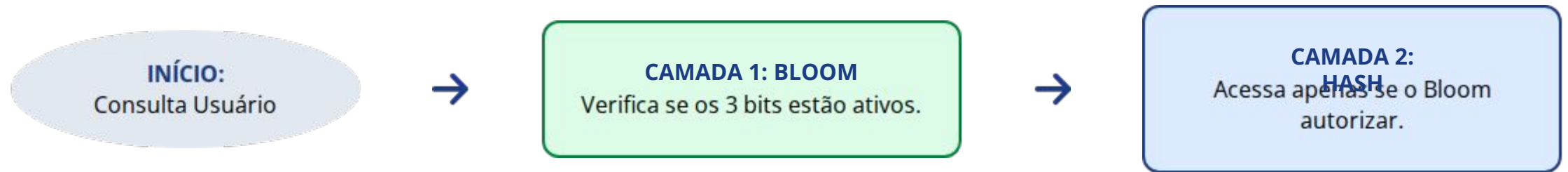
Natureza Probabilística

O filtro utiliza um **vetor de bits** denso para representar a presença de dados. Ele nunca gera "falsos negativos" (se diz que não existe, é certeza). No entanto, pode gerar **falsos positivos** devido à sobreposição de bits ativados por diferentes chaves.

Parâmetros de Construção

- **Vetor de Bits:** 900.000 posições estáticas.
- **Funções Hash (k=3):** Três funções independentes mapeiam cada string para 3 índices diferentes no
- **Bitset:** Ocupa significativamente menos memória que a Tabela Hash original.

Como as Estruturas Cooperam



A Lógica do "Gatekeeper": O Filtro de Bloom intercepta consultas de elementos que não estão no banco de dados. Isso evita o cálculo de hash complexo e o acesso à memória da Tabela Hash, economizando recursos computacionais preciosos.

Organização e Compilação

Arquitetura Modular

O código foi dividido logicamente para garantir manutenibilidade e clareza:

- **main.c:** Interface com o usuário e execução dos testes automatizados.
- **hash.c / hash.h:** Implementação da Tabela Hash djb2.
- **bloom.c / bloom.h:** Manipulação de bits e funções $k=3$.

```
// Comando de Compilação GCC
```

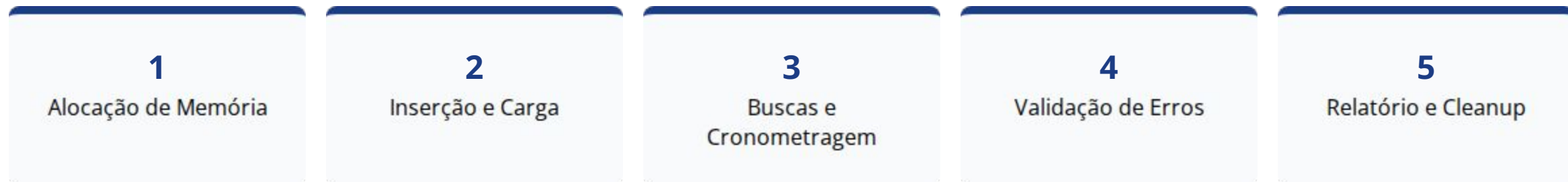
```
gcc main.c hash.c bloom.c -o sistema
```

```
// Execução
```

```
./sistema
```

O Procedimento Experimental

Para cada cenário (1k, 10k, 100k), seguimos um rigoroso protocolo de cinco etapas:



*A cronometragem foi realizada utilizando a biblioteca **time.h**, medindo o tempo de processamento em milissegundos para garantir precisão científica.*

Resultados Consolidados

Cenário (N)	Colisões Hash	Tempo sem Bloom (s)	Tempo com Bloom (s)	Falsos Positivos Reais
1.000 (1k)	0	0,000	0,000	0,10%
10.000 (10k)	0	0,003	0,002	0,87%
100.000 (100k)	0	0,026	0,025	8,97%

Os dados mostram que a Tabela Hash operou perfeitamente (0 colisões), permitindo que qualquer variação de tempo fosse atribuída exclusivamente à eficiência do Filtro de Bloom.

O cenário de 1.000 registros atingiu o limite de precisão do relógio do sistema (arredondando para zero).

Ganhos de Velocidade e Eficiência

33,3%

Redução de Tempo (10k)

No cenário de 10.000 elementos, o tempo de busca caiu de 0,003s para 0,002s. Este foi o ponto de **máxima eficiência** do sistema.

3,8%

Ganho Marginal (100k)

No cenário de carga máxima, o ganho percentual diminuiu. O sistema ainda é mais rápido com Bloom, mas o custo dos erros começa a pesar.

Por que o ganho diminuiu? Com o aumento da taxa de falsos positivos (8,9%), o sistema "falhou" em descartar quase 9% das chaves inexistentes, obrigando a busca redundante na Tabela Hash e gerando um **overhead**.

Teoria vs. Prática no Filtro de Bloom

A taxa de erro do Filtro de Bloom segue uma curva assintótica. À medida que mais elementos são inseridos, mais bits '1' são marcados no vetor.

$$p \approx (1 - e^{-k \cdot n / m})^k$$

Onde **m** = 900.000 bits e **k** = 3 funções hash.

Saturação do Vetor

- **1k e 10k:** Abundância de bits '0'. A chance de colisão acidental é ínfima (< 1%).
- **100k:** O vetor atinge uma densidade crítica. A sobreposição de bits ativos causa o salto para 8,97% de erro.

Isso prova que a eficácia do Bloom depende da proporção **bits/elemento**.

Análise das Perguntas (Parte I)

1. O Filtro de Bloom reduziu consultas na Tabela Hash?

Sim. Os testes de cronometragem confirmam que, ao identificar elementos como inexistentes, o algoritmo encerra a busca imediatamente, poupando a Tabela Hash de processar quase 100% das chaves inválidas nos cenários 1k e 10k.

2. Como o tamanho do vetor impactou os resultados?

O vetor de 900.000 bits foi excelente para cargas baixas. Porém, demonstrou que a estrutura **não é infinitamente escalável**. No cenário de 100k, a densidade de bits '1' degradou a precisão, mostrando que para volumes maiores, seria necessário redimensionar o vetor dinamicamente.

Análise das Perguntas (Parte II)

3. O número de funções hash alterou a precisão?

Sim. $k=3$ foi o equilíbrio projetado. Se usássemos menos, teríamos mais colisões por falta de assinatura. Se usássemos mais (ex: $k=7$), o vetor de bits saturaria muito mais rápido, degradando a performance para volumes de dados menores.

4. Qual o cenário mais vantajoso para o uso do Bloom?

O cenário de **10.000 registros**. Nele, conseguimos o maior equilíbrio: redução expressiva de 33% no tempo de busca com uma taxa de erro residual desprezível (0,87%). Foi o "sweet spot" técnico do nosso dimensionamento.

Conclusões do Projeto

O projeto validou com sucesso a integração de estruturas **determinísticas** e **probabilísticas** para otimização de consultas.

- A Tabela Hash dimensionada com fator 0,5 garante busca estável $O(1)$.
- Estruturas probabilísticas são ferramentas poderosas para redução de latência em sistemas de larga escala.

Veredito Final

A performance de um sistema de software não é determinada apenas pelo hardware, mas pela **escolha e ajuste inteligente dos algoritmos**. O Filtro de Bloom provou ser um aliado indispensável para a escalabilidade de bases de dados.

Referências e Bibliografia

SINGH, A. P. Bloom filters. AlgoMaster, 2024. [blog.algomaster.io]

SCHONSS, S. Tabelas hash. Medium, 2024. [medium.com/@sschonss]

PIMENTEL, R. Tabela hash. UFU, 2025. [facom.ufu.br]

GOOGLE. Introducing Gemini: our most capable AI model yet. Google AI, 2023.

UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO (UFERSA)

Bacharelado em Computação | Pau dos Ferros-RN